VISUALIZATION

Visualization is a way of representing data in a graphical or pictorial format for better understanding of data. Visualization gives a good idea of data and the trends within it.

In Python, data visualization has multiple libraries like matplotlib, seaborn, plotly, etc.

Let us dive deeper into matplotlib and seaborn.

MATPLOTLIB

Matplotlib, introduced in 2002 by John Hunter, is used for plotting 2D plots and is built on NumPy.

Basic components of matplotlib:

Figure: The entire window or page where plots are drawn.

Axes: The area where data is plotted (can be multiple in one figure).

Axis: The number lines (x, y) that frame the data.

Labels: Used for naming axes and the plot.

Legends: Helps distinguish multiple datasets in the same graph.

Title: Gives a heading to the plot.

Installation

```
pip install matplotlib
```

Importing

```
import matplotlib.pyplot as plt
```

3D Surface Plot

3D surface plots help visualize the relationship between three continuous variables. They're great for showing complex mathematical surfaces.

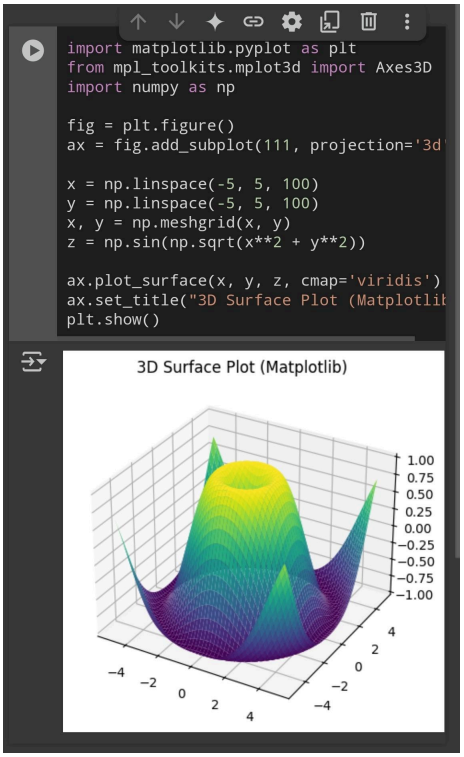
Code Snippet & Output :

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
x, y = np.meshgrid(x, y)
z = np.sin(np.sqrt(x**2 + y**2))

ax.plot_surface(x, y, z, cmap='viridis')
ax.set_title("3D Surface Plot (Matplotlib)")
plt.show()
```



Description: This graph displays a wavy surface generated from sine of the distance from origin, creating a ripple effect in 3D.

Polar Plot

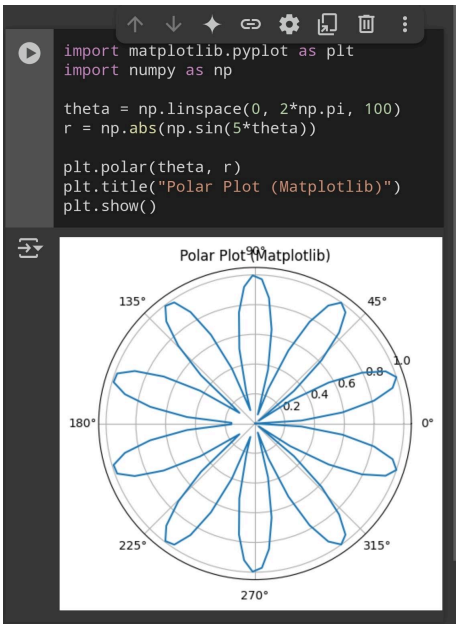
Polar plots represent data in circular form using angles and radius.
They're ideal for visualizing periodic functions or cyclic patterns.

Code Snippet & Output :

```
import matplotlib.pyplot as plt
import numpy as np

theta = np.linspace(0, 2*np.pi, 100)
r = np.abs(np.sin(5*theta))

plt.polar(theta, r)
plt.title("Polar Plot (Matplotlib)")
plt.show()
```



Description: The graph forms a flower-like shape with 5 petals based on the sine function of angle theta.

Pie Chart

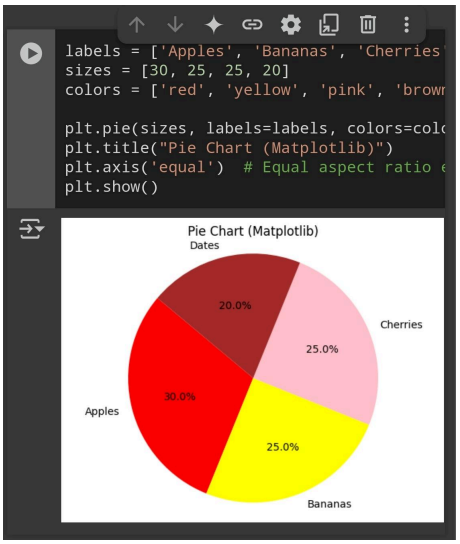
Pie charts are used to display parts of a whole as slices.
They're useful for showing percentage or proportion data.

Code Snippet & Output :

```
import matplotlib.pyplot as plt

labels = ['Apples', 'Bananas', 'Cherries', 'Dates']
sizes = [30, 25, 25, 20]
colors = ['red', 'yellow', 'pink', 'brown']

plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=140)
plt.title("Pie Chart (Matplotlib)")
plt.axis('equal')  # Equal aspect ratio ensures the pie is circular
plt.show()
```



Description: The chart displays fruit proportions: Apples (30%), Bananas (25%), Cherries (25%), and Dates (20%) in colorful slices.

Error Bar Plot

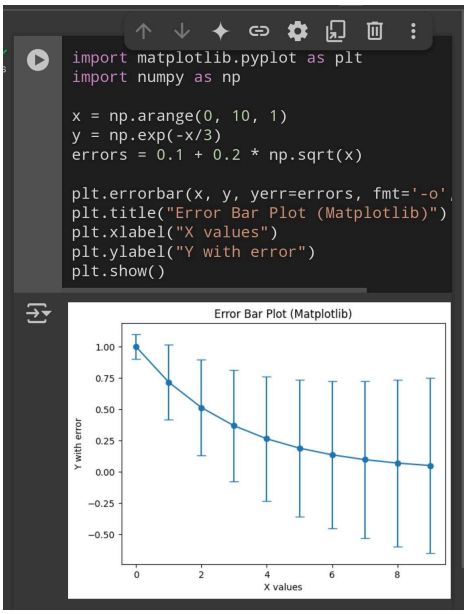
Error bar plots show uncertainties or variations in data.
They're great for displaying experimental data with margins of error.

Code Snippet & Output :

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0, 10, 1)
y = np.exp(-x/3)
errors = 0.1 + 0.2 * np.sqrt(x)

plt.errorbar(x, y, yerr=errors, fmt='-o', capsize=5)
plt.title("Error Bar Plot (Matplotlib)")
plt.xlabel("X values")
plt.ylabel("Y with error")
plt.show()
```



Description: This graph visualizes an exponential decay function with vertical error bars showing uncertainty in the y values.

Stacked Bar Plot

Stacked bar plots compare total values and their sub-components.
They're useful for visualizing group-wise contributions.

Cide Snippet & Output :

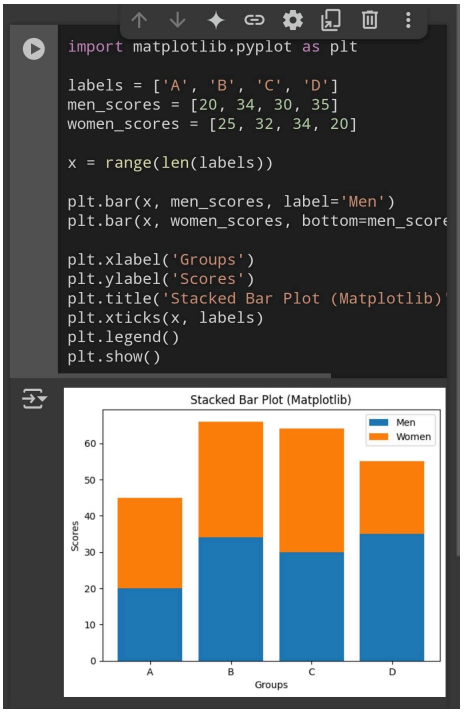
```
import matplotlib.pyplot as plt

labels = ['A', 'B', 'C', 'D']
men_scores = [20, 34, 30, 35]
women_scores = [25, 32, 34, 20]

x = range(len(labels))

plt.bar(x, men_scores, label='Men')
plt.bar(x, women_scores, bottom=men_scores, label='Women')

plt.xlabel('Groups')
plt.ylabel('Scores')
plt.title('Stacked Bar Plot (Matplotlib)')
plt.xticks(x, labels)
plt.legend()
plt.show()
```



Description: This chart compares scores of men and women across groups A to D by stacking their values for total height.

SEABORN

Seaborn is a powerful Python data visualization library built on top of Matplotlib. It provides a high-level interface for creating attractive and informative statistical graphics. It is especially useful when working with Pandas DataFrames.

Basic Components of Seaborn:

Themes: Controls the overall look (e.g., darkgrid, whitegrid, etc.).

Color Palettes: Predefined color styles for consistency and beauty.

Statistical Plots: Includes box plots, violin plots, KDE plots, etc.

Built-in Datasets: Comes with datasets like tips, iris, titanic.

Integration: Seamlessly integrates with Pandas and NumPy.

Installation:

```
pip install seaborn
```

Importing:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Heatmap

Visualizes correlation between multiple variables using color intensity.
Great for analyzing relationships between features in a dataset.

Code Snippet & Output :

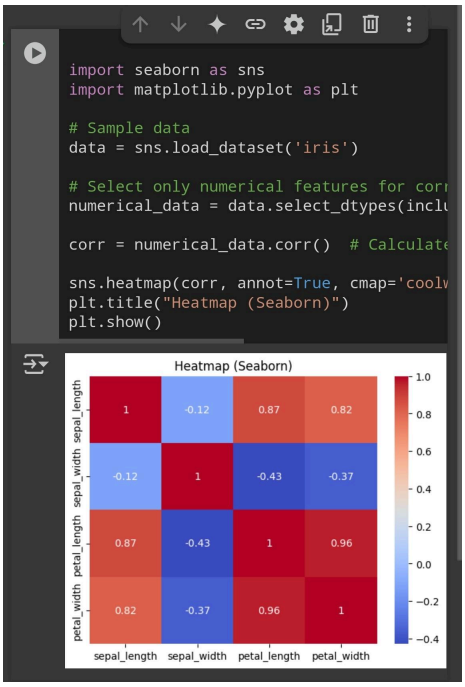
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = sns.load_dataset('iris')

# Select only numerical features for correlation calculation
numerical_data = data.select_dtypes(include=['number'])  # Exclude non-numeric columns

corr = numerical_data.corr()  # Calculate correlation on numerical data only

sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Heatmap (Seaborn)")
plt.show()
```



Description:
This heatmap shows the correlation between numerical columns in the dataset using different color intensities — darker shades mean higher correlation.

Violin Plot

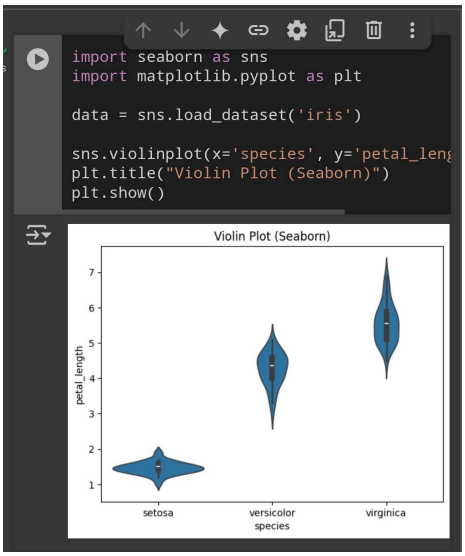
Combines a box plot and KDE plot to show distribution and probability.
Useful for visualizing the spread and skewness of data.

Code Snippet & Output :

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset('iris')

sns.violinplot(x='species', y='petal_length', data=data)
plt.title("Violin Plot (Seaborn)")
plt.show()
```



Description:
This violin plot shows how petal lengths vary for different Iris species — wide sections represent higher data density.

Pair Plot

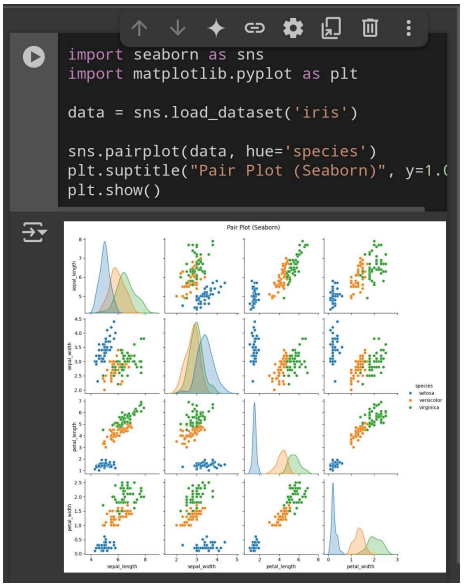
Creates a grid of scatter plots to show pairwise relationships.
Useful for quickly exploring multiple features.

Code Snippet & Output :

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset('iris')

sns.pairplot(data, hue='species')
plt.suptitle("Pair Plot (Seaborn)", y=1.02)
plt.show()
```



Description:
Each scatter plot compares a pair of features, and diagonal plots show distributions. Colors represent different species.

Swarm Plot

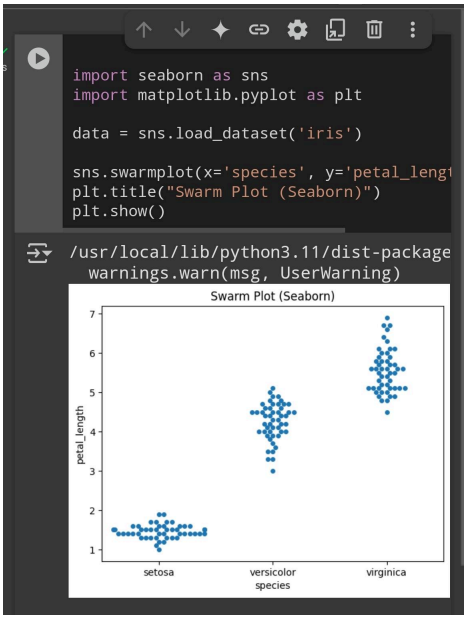
Shows all data points while avoiding overlap — like a better scatter for categories.
Good for seeing distribution and clusters.

Code Snippet & Output :

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset('iris')

sns.swarmplot(x='species', y='petal_length', data=data)
plt.title("Swarm Plot (Seaborn)")
plt.show()
```



Description:
This plot shows how petal length differs among species, with each dot representing a data point. It's more readable than standard scatter plots for categories.

Boxen Plot

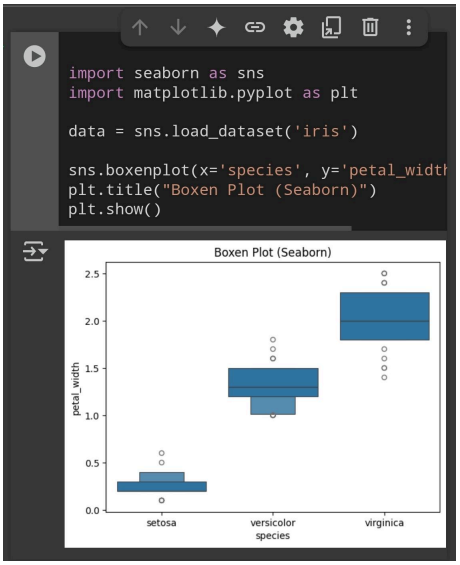
Enhanced box plot for large datasets, showing more quantiles.
Great for visualizing skewed distributions.

Cide Snippet & Output :

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset('iris')

sns.boxenplot(x='species', y='petal_width', data=data)
plt.title("Boxen Plot (Seaborn)")
plt.show()
```



Description:
This plot gives a detailed look at petal width for different species, especially for larger datasets with heavy tails.